Pointfree CoEndoYoneda Lemma in Functional Categories Theory and Formalization in Lean 4

Luc Duponcheel Antigravity AI

July 25, 2026

Abstract

We present a variation of the CoYoneda Lemma for functional categories equipped with a monadic structure of its global elements, which we formally identify as *Functional Categories*. The central theoretical contribution, the Pointfree CoEndoYoneda Equivalence, establishes an isomorphism between, on the one hand, natural transformations mapping out of co-Yoneda endofunctors to a designated kind of global endofunctors and, on the other hand, some corresponding kind of global elements.

The original motivation was to formulate an equivalence that could play a role in the broader programming world, encompassing both pure programs, more precisely functions, and impure programs, more precisely computation with side-effects valued functions (modeled via Kleisli categories). While the requirements proved too restrictive to admit arbitrary monadic effects, this limitation reveals a profound mathematical result: the *Functional Category* constraints categorically classify the essence of referential transparency.

In addition to outlining the mathematical foundations, we detail the complete formalization of this result in the Lean 4 theorem prover. We discuss the technical challenges of working with functor compositions under strict definitional equality and demonstrate strategies for navigating dependent type theory to construct fully formal, pointfree proofs of higher-level category theory results.

1 Introduction

The *CoYoneda Lemma* serves as a cornerstone of modern category theory, asserting that for any category $\mathcal{C}$, object $X \in \mathcal{C}$, and functor $F : \mathcal{C} \to \mathbf{Set}$, natural transformations $\mathrm{Nat}(\mathcal{C}(X, -), F)$ are isomorphic to functor evaluations $F(X)$. This result fundamentally states that an object X is completely determined by the morphisms out of it.

In computational settings, we frequently encounter categories equipped with additional structure. Categories that have a functor mapping functions of types (or sets) to programs, their images being called pure programs. We define such a category as a *Functional Category*. In this paper, we explore how a variation of the CoYoneda isomorphism extends in this setting. The variation replaces elements by global elements and is entirely pointfree.

A key motivation for this exploration is the world of computer programs, which consist of both "pure programs without side-effects" and "impure programs with side-effects". We set out to formulate a CoEndoYoneda equivalence for endofunctors that could model this broader programming world using monads. Remarkably, we discover that the structural requirements of our equivalence serve as a sieve for purity, mathematically forcing monadic referential transparency.

We provide a comprehensive discussion of how these concepts and theorems are formalized in the Lean 4 proof assistant using the `Mathlib` library.

2 Functional Categories

Let $\mathcal{C}$ be a locally small category. We denote the category of types (or sets) as **Type**. For any object $X \in \mathcal{C}$, the covariant Hom-functor, which we will call the CoYoneda functor, is defined as:

$$CYF_X : \mathcal{C} \to \mathbf{Type}, \quad Y \mapsto \mathcal{C}(X, Y)$$

Definition 2.1. *A **Functional Category** is a category $\mathcal{C}$ equipped with the following data:*

1. *A designated functor $\Phi : \mathbf{Type} \to \mathcal{C}$.*
2. *A constructed endofunctor $GF : \mathcal{C} \to \mathcal{C}$ defined by $GF(X) = \Phi(CYF_{\Phi(1)}(X))$.*
3. *A monadic structure on the globals of $\mathcal{C}$, provided by a unit natural transformation $\gamma\eta : Id_{\mathcal{C}} \to GF$ and a multiplication natural transformation $\gamma\mu : GF \circ GF \to GF$.*

These structural components must satisfy the standard monad left unit law:

$$\gamma\eta_{GF(X)} \circ \gamma\mu_X = id_{GF(X)}$$

For now no other extra monad laws are needed in the formal Lean 4 proof. This is an interesting observation on its own.

This structure essentially allows $\mathcal{C}$ to reflect the category of types into itself, while treating GF as a monadic context. We define the *CoYoneda EndoFunctor* $CYEF_X : \mathcal{C} \to \mathcal{C}$ for a fixed object X as the composition:

$$CYEF_X = \Phi \circ CYF_X$$

We use the following definition of *global element* as a morphisms from the canonical singleton $\Phi(1)$ to X.

$$G_\Phi(X) = \mathcal{C}(\Phi(1), X)$$

2.1 Formalization of Functional Categories in Lean 4

In Lean 4, a functional category is modeled as a type class extending the standard `LargeCategory` from `Mathlib`.

```
abbrev CYF {C : Type (u + 1)} [LargeCategory.{u} C] (X : C) : C ⥤ Type u :=
  coyoneda.obj (op X)

abbrev CYEF_def
  {C : Type (u + 1)} [LargeCategory.{u} C] (Φ : Type u ⥤ C) (X : C) : C ⥤ C :=
    CYF X ⋙ Φ

abbrev GEF_def {C : Type (u + 1)} [LargeCategory.{u} C] (Φ : Type u ⥤ C) : C ⥤ C :=
  CYEF_def Φ (Φ.obj PUnit)

class FunctionalCategory (C : Type (u + 1)) extends LargeCategory.{u} C where
  Φ : Type u ⥤ C

  γμ : (GEF_def Φ ⋙ GEF_def Φ) ⟶ GEF_def Φ
  γη : 1 ⟶ GEF_def Φ

  γ_left_unit :
    ∀ (X : C), γη.app ((GEF_def Φ).obj X) ≫ γμ.app X = id ((GEF_def Φ).obj X)

  φ {X Y : Type u} (f : X → Y) : Φ.obj X⟶Φ.obj Y := Φ.map (TypeCat.ofHom f)

  φ_eq :
    ∀ {X Y : Type u} (f : X → Y), φ f = Φ.map (TypeCat.ofHom f) :=
      by intros; rfl

  γη_φ :
    ∀ (X : Type u), γη.app (Φ.obj X) = φ (fun x ⇒ φ (fun _ ⇒ x)) :=
      by intros; rfl
```

and global elements are modeled as

```
abbrev G
  {C : Type (u + 1)} [LargeCategory.{u} C] (Φ : Type u ⥤ C) (X : C) : Type u :=
    Φ.obj PUnit⟶X
```

A canonical, trivial instance of the type class is the world of pure programs without side-effects (functions), **Type** itself, where Φ is the identity functor.

```
instance typesFunctionalCategory : FunctionalCategory (Type u) where
  Φ := 1 (Type u)
  γη := {
    app := fun X ⇒ (1 (Type u)).map (↑(fun x ⇒ ↑(fun _ ⇒ x)))
    naturality := fun X Y f ⇒ by ext; rfl
  }
  γμ := {
    app := fun X ⇒ (1 (Type u)).map (↑(fun (f : PUnit⟶PUnit⟶X) ⇒ ↑(fun pu ⇒ f pu
          pu)))
    naturality := fun X Y f ⇒ by ext; rfl
  }
  γ_left_unit := fun _ ⇒ by rfl
```

3 The CoEndoYoneda Equivalence

The requirements of `FunctionalCategory` prove to be sufficient to establish a CoEndoYoneda equivalence for endofunctors.

3.1 Auxiliary definitions and theorems

We construct a designated global element from a designated natural transformation as follows

```
def τX2ggfx {F : C ⥅ C} {X : C} (τX : CYEF X ⟶ F ⋙ GEF) :
  Global ((F ⋙ GEF).obj X) :=
    let ggfx := φ (fun _ ⇒ id X) ≫ τX.app X
    ggfx
```

We construct a designated (natural, not shown) transformation from a designated global element as follows

```
def ggfx2τX
  {F : C ⥅ C} {X : C} (ggfx : Global (GEF.obj (F.obj X))) : CYEF X ⟶ (F ⋙ GEF) :=
    let τX : CYEF X ⟶ F ⋙ GEF := {
      app Y := φ (ggfx ≫ (F ⋙ GEF).map .) ≫ γμ.app (F.obj Y)
      naturality _ _ h := ...
    }
    τX
```

We formulate (and prove, not shown) a left inverse theorem as follows

```
theorem left_inverse {F : C ⥅ C} {X : C} (τX : CYEF X ⟶ (F ⋙ GEF)) :
    τx = ggfx2τx (τx2ggfx τx) := by ...
```

We formulate (and prove, not shown) a right inverse theorem as follows

```
theorem right_inverse {F : C ⥅ C} {X : C} (ggfx : Global ((F ⋙ GEF).obj X)) :
    ggfx = τx2ggfx (ggfx2τx ggfx) := by ...
```

Notice that both inverses apply φ (defined as Φ.map) to fun $\_ \Rightarrow \mathrm{id}_X$, which mirrors the classic CoYoneda construction.

3.2 CoEndoYoneda Equivalence

Utilizing the auxiliary definitions and theorems above we can now define the main equivalence.

```
def coEndoYonedaEquiv {F : C ⥅ C} {X : C} :
  (CYEF X ⟶ (F ⋙ GEF)) ≃ Global ((F ⋙ GEF).obj X) where
  toFun    := τX2ggfx
  invFun   := ggfx2τX
  left_inv  := fun τX ⇒ (left_inverse  τX).symm
  right_inv := fun τX ⇒ (right_inverse τX).symm
```

This equivalence provides a completely pointfree and logically verified method for evaluating and constructing natural transformations in functional categories.

4 The Essence of Purity (Every Disadvantage Has an Advantage)

The original motivation was to define a CoEndoYoneda equivalence that could play a role in the broader programming world, which consists of both pure programs (functions) and impure programs (computation with side-effects valued

functions). The natural setting for such impure programs is the Kleisli category of a Monad M, denoted `KleisliCat M`.

However, attempting to instantiate `KleisliCat M` as a `FunctionalCategory` reveals an apparent limitation. The equivalence cannot be instantiated for arbitrary monads. Thus, the `coEndoYonedaEquiv` is not universally useful for the (potentially impure) programming world.

But this disadvantage reveals a profound mathematical advantage:
the `FunctionalCategory` requirements, more precisely $\gamma\eta$-naturality, act as a strict classification of purity for `KleisliCat M`. They mathematically declare the essence of referential transparency of monad computations (cfr. referential transparency of expression evaluations).

4.1 The Purity Constraint

The core constraint imposed by $\gamma\eta$.naturality demands that mapping `pure` over the structure of a monadic value m is exactly the same as wrapping the entire structure m in `pure`. Formally, it forces the equality:

$$m \gg\!= \lambda z.\text{pure}(\text{pure}(z)) = \text{pure}(m)$$

If a Monad satisfies this purity constraint (which is strictly required to instantiate `FunctionalCategory` via `KleisliCat`), then its shape functor evaluated at `PUnit` is a subsingleton (proof not shown).

```
def EnforcesPurity : Prop :=
  ∀ {X : Type v} (mx : M X), (mx >>= fun x ⇒ pure (pure x)) = pure mx

theorem purity_implies_subsingleton_shape
  (h_pure : EnforcesPurity M) : Subsingleton (M PUnit) := by ...
```

This implies that M is a "Subsingleton Monad" (such as the Identity Monad or the Terminal Monad). In other words, any standard computational effect is aggressively filtered out by the type system. We have formally verified this rejection for six standard computational monads: `Option`, `List`, `Reader`, `Writer`, `State`, and `Continuation`. Satisfying the Purity constraint for these monads either leads to a direct logical contradiction (as with `Option` and `List`) or mathematically forces their underlying parameters to be trivial (e.g. forcing the state type or the reader environment to be a `Subsingleton`).

We formalize this conclusion explicitly (proof not shown):

```
theorem functional_category_implies_purity
  (γη : 1 (KleisliCat M) ⟶ (GEF_def (κΦ M)))
  (h_γη_app : ∀ X, γη.app X = fun x ⇒ pure (fun _ ⇒ pure x)) :
  EnforcesPurity M := by ...
```

Referential transparency is exactly the requirement for a computational context to admit a global coyoneda endofunctor equivalence.

5 Proof Techniques and Challenges in Lean 4

Formalizing abstract category theory in dependent type theory (DTT) introduces unique difficulties, particularly concerning definitional equality.

5.1 Strict Functor Composition

In standard mathematics, functor composition is strictly associative. In Lean 4, however, these are represented as distinct abstract syntax trees. They are *isomorphic* via `Category.assoc`, but they are not *definitionally equal*.

When writing the proof of `left_inverse`, this limitation prevents standard term-rewriting tactics like `simp` from effortlessly reducing deeply nested compositions.

5.2 Constructing Equality Chains

To circumvent the strict type-checker without resorting to manual casts (such as `eqToHom`), the proof for `left_inverse` abandons the monolithic `simp` approach in favor of explicit equality chains acting on the natural transformation components.

By utilizing `Eq.trans` and `congr_arg`, we explicitly guide the type checker through the unit laws and naturality squares:

```
have h1 :
  τX.app Y = τX.app Y ≫ id ((GEF_def Φ).obj (F.obj Y)) :=
    (Category.comp_id (τX.app Y)).symm
have h2 :
  id ((GEF_def Φ).obj (F.obj Y)) =
    γη.app ((GEF_def Φ).obj (F.obj Y)) ≫ γμ.app (F.obj Y) :=
      (γ_left_unit (F.obj Y)).symm
have h_left_unit :
  τX.app Y =
    τX.app Y ≫ (γη.app ((GEF_def Φ).obj (F.obj Y)) ≫ γμ.app (F.obj Y)) :=
      Eq.trans h1 (congr_arg (τX.app Y ≫ .) h2)
```

This localized approach bypasses the rigid typing of the category morphisms by acting directly on the equality type `=` in the universe of types. This technique is highly robust and avoids the infamous "motive is not type correct" errors common in Lean category theory formalization.

6 Conclusion

The Pointfree CoEndoYoneda Lemma is a variation of the classical Pointful CoYoneda Lemma in a functional category setting. While our initial aim was to generalize this equivalence to arbitrary computational effects via monads, the rigorous formalization process revealed that the required categorical structure fundamentally filters out impurities.

By successfully formalizing the lemma in Lean 4, we have not only verified its absolute logical correctness but also established that `FunctionalCategory` mathematically classifies referential transparency. Moreover, we have demonstrated a concrete methodology for handling strict composition equality in dependent type theory. The approach of combining explicit structural definitions

with targeted, step-by-step equality chaining provides a reliable method for formalizing advanced category theory in modern proof assistants.